Module 2 — Dark Social Intelligence Engine: Implementation Plan

What we're building and why
Module 2 answers: "Where is piracy happening and who is behind it?" Unlike Module 1 (content matching), Module 2 tracks piracy in private spaces like Telegram, WhatsApp, and Discord by monitoring their **public traces** — invitation links, new domains, torrents, and aggregator sites.

The system has **5 components** + **1 central intelligence layer** that connects everything into a threat picture.

Component 1 — Link Surface Crawler
Purpose: Automatically scans aggregator websites (like "soccerstreams") that list illegal sports streams.

How it works:

- Starts from known seed sites, loads full JavaScript-rendered pages
- Finds streaming URLs with patterns like "live", "stream", ".m3u8"
- Follows internal/external links (3 levels deep) to discover new sites
- Runs every 5 min during events, hourly otherwise (with delays to avoid blocks)

Output: Records source site, stream URL, timestamp, context → feeds threat graph

---

Component 2 — Invite Link Harvester
Purpose: Collects public invitation links to private piracy groups posted on Twitter, Reddit, public Discord.

How it works:

- **Twitter/X**: API searches for sports terms + invite patterns (t.me/, discord.gg/, chat.whatsapp.com/)
- **Reddit**: Monitors sports subreddits' public posts/comments
- **Discord**: Joins public sports servers, reads public channels only
- Records poster account, reach (followers/retweets), timestamp

Output: Platform, account, invite link, reach, destination → connects public accounts to private groups

---

Component 3 — DNS Shadow Tracker
Purpose: Detects new piracy domains **before** they go live using Certificate Transparency logs.

How it works:

- Monitors real-time WebSocket feed of all new HTTPS certificates
- Flags domains with piracy patterns:
    1. Keywords: "stream", "live", "sport" + "free", "HD"
    2. Known piracy brand names (ex: "streameast")
    3. Cheap TLDs: .xyz, .top, .site, .online
- Checks WHOIS, certificate issuer, if site is live

Output: Domain, cert date, risk score, patterns matched → new domain nodes

---

Component 4 — Torrent DHT Monitor
Purpose: Passively monitors BitTorrent DHT network for sports torrents without downloading content.

How it works:

- Joins DHT via bootstrap nodes
- Watches "announce-peer" messages (torrent info hashes + IP addresses)
- Requests torrent metadata (name/size) via protocol extension
- Flags sports-related names: league/team names, "match", "highlights"
- Analyzes if IP is datacenter/hosting provider

Output: Info hash, torrent name, size, IP, country, datacenter status → connects torrents to IPs

---

Component 5 — Threat Graph (Intelligence Layer)
Why a graph? Individual data points are limited. Connections reveal organized networks.

Graph structure:

- **Nodes**: Actors (accounts), domains, stream links, invite links, torrents, IPs
- **Edges**: "posted", "hosts", "shared", "leads to", "seeded by", "resolves to"

How it's built:

- All 4 components write to Neo4j graph database + PostgreSQL (raw data)
- Same actor → adds edges, doesn't duplicate nodes
- Redis/Celery queues manage workload spikes

Analysis algorithms:

1. **PageRank** (every 15 min): Finds central "hub" actors
2. **Louvain community detection** (every 2 hrs): Identifies coordinated piracy groups

Output: Ranked high-threat actors + full connection history for dashboard

---

End-to-end workflow:

text

```
4 crawlers → Redis/Celery queue → Neo4j graph + PostgreSQL
↓
PageRank (15min) + Community detection (2hrs)
↓
FastAPI → Rights dashboard (visual graph, actor traces, evidence export)
```

Dashboard shows: Live threat graph (node size = PageRank), click nodes for full history across all sources.